

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

One more example on data streams

Haskell `iterate`

Corecursion

Challenge: "quasi-static" Tiki Taka

Tiki Taka style in football

Tiki Taka style consists of quick passing and movement.

Challenge

Two players, C and D, are training on a standard pitch (105×68 meters). They have to practice Tiki Taka in a "quasi-static" manner: the game starts with one of the players, say C, having the ball. D makes no move and waits for the ball. Now C passes the ball to D, then runs to a random point, anywhere on the pitch. When D receives the ball, C stops and D passes it to C, in the precise position C is situated at that moment. Now D starts running and stops when C passes the ball to him, and so on. Come up with an assessment of the average length of a pass. You should either come with a Haskell implementation, or with a mathematical proof.

A circular structure for generating primes

Example (Successsively eliminate multiples of primes)

```
primes =  
  let  
    oPrs n | isPr n primes  = n:oPrs (n+2)  
           | otherwise      = oPrs (n+2)  
    isPr n (x:xs) | n < x^2      = True  
                  | n `mod` x == 0 = False  
                  | otherwise     = isPr n xs  
  in  
    2:oPrs 3
```

A circular structure for generating primes

Example (A trace of primesGen)

```
= 2:oPrs 3      [isPr 3 (2: ... )    = True]

= 2:3:oPrs 5     [isPr 5 (2:3: ... ) =
                  isPr 5 (3: ... )   = True]

= 2:3:5:oPrs 7   [isPr 7 (2:3: ... ) =
                  isPr 7 (3: ... )   = True]

= 2:3:5:7:oPrs 9 [isPr 9 (2:3: ... ) =
                  isPr 9 (3: ... )   = False]

= 2:3:5:7:oPrs 11 [isPr 11 (2:3:5 ... ) =
                   isPr 11 (3:5 ... )   =
                   isPr 11 (5..)       = True]

= ...
```

Iteratively calling a function in Haskell

Definition (Haskell function `iterate`)

```
iterate :: (a -> a) -> a -> [a]  
iterate f x = x : iterate f (f x)
```

Iterates the call of `f` and builds the infinite sequence

`[x, f(x), f(f(x)), ...]`

Example (Haskell function `iterate`)

```
iterate (*2) 1  
= 1 : iterate (*2) ((*2) 1)  
= 1 : 2 : iterate (*2) ((*2) 2)  
= 1 : 2 : 4 : iterate (*2) ((*2) 4)  
= ...
```

Iteratively calling a function in Haskell

Haskell function `iterate`

```
iterate :: (a -> a) -> a -> [a]  
iterate f x = x : iterate f (f x)
```

Example (Haskell function `iterate`)

```
Prelude> take 5 $ iterate tail [1,2,3,4]  
[[1,2,3,4],[2,3,4],[3,4],[4],[]]  
Prelude> take 5 $ iterate ((^2) . (+1) . sqrt) 1.0  
[1.0,4.0,9.0,16.0,25.0]
```

Computing $\sqrt{a}$ iteratively

Example (The "Babylonian method" for $\sqrt{a}$)

start with an initial approximation x_0

repeat

$$x_{k+1} = (a / x_k + x_k) / 2$$

until a stopping criterion is met

(e.g. a given number of iterations is reached *or*

2 consecutive approximations are "very close" to each other)

```
nextap :: Double -> Double -> Double
```

```
nextap a capx = (a / capx + capx) / 2.0
```

Computing $\sqrt{a}$ iteratively

Example (Approximating $\sqrt{3}$)

```
nextap :: Double -> Double -> Double
nextap a capx = (a / capx + capx) / 2.0
```

We arbitrarily set the initial approximation value (`capx`) to `1.0`.
A manual trace of the first iterations will give:

```
*Main> nextap 3.0 1.0
2.0
*Main> nextap 3.0 2.0
1.75
*Main> nextap 3.0 1.75
1.7321428571428572
```

Computing $\sqrt{a}$ iteratively

Example (Approximating $\sqrt{3}$)

```
nextap :: Double -> Double -> Double
```

```
nextap a capx = (a / capx + capx) / 2.0
```

```
*Main> take 5 $ iterate (nextap 3.0) 1.0
```

```
[ 1.0,  
  2.0,  
  1.75,  
  1.7321428571428572,  
  1.7320508100147274 ]
```

Computing $\sqrt{a}$ iteratively

Definition (Stopping criterion: "very close" approximations)

Instead of stopping after a given number of iterations, we'll stop when the difference between two consecutive terms is smaller, in absolute value, than a preset value ϵ (they are "very close").

```
stopIt eps (ac:an:as)
  | veryClose eps ac an = an
  | otherwise           = stopIt eps (an:as)
where
  veryClose eps a1 a2 = abs (a1-a2) < eps
```

Computing $\sqrt{a}$ iteratively

Example (Approximating $\sqrt{3}$)

```
nextap a capx = (a / capx + capx) / 2.0
```

```
stopIt eps (ac:an:as)  
  | veryClose eps ac an = an  
  | otherwise           = stopIt eps (an:as)  
where  
  veryClose eps a1 a2 = abs (a1-a2) < eps
```

```
aproxSqrt a eps =  
  stopIt eps (iterate (nextap a) 1.0)
```

```
*Main> aproxSqrt 3 10E-5  
1.7320508100147274
```

PageRank algorithm

Definition (PageRank score of node in a directed graph)

$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

where:

$p_1, \dots, p_N$ are graph nodes (=pages)

N is the number of nodes in the graph

$M(p_i)$ is the set of nodes that send links to p_i

$L(p_j)$ is the number of outbound links of node p_j

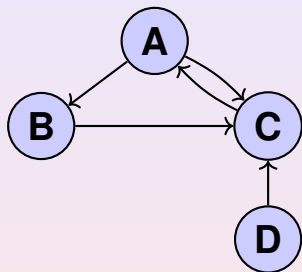
d is a damping factor (usually 0.85)

Remark

The score of node N depends on the scores of the nodes sending links towards N .

PageRank algorithm

Example



$$PR(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

PageRank algorithm

Example

```
type Name = String
type Degree = Int
type Node = (Name, [Name], Degree)
type Graph = [Node]
type Score = Double
type Ranks = [(Name, Score)]

d = 0.85 :: Double --dampening factor
dimG = 4 :: Int --number of nodes in the graph
prec = 10E-5 :: Double --convergence precision
```

PageRank algorithm

Example

```
outDegree :: Name -> Graph -> Degree
```

```
outDegree n ((c,_,d):nds)
  | n==c      = d
  | otherwise = outDegree n nds
```

```
nodesIn :: Name -> Graph -> [Name]
```

```
nodesIn n ((c,ins,_):nds)
  | n==c      = ins
  | otherwise = nodesIn n nds
```

```
nodeRank :: Name -> Ranks -> Score
```

```
nodeRank n ((c,r):ranks)
  | n==c      = r
  | otherwise = nodeRank n ranks
```

PageRank algorithm

Example

```
nodeUpdate :: Name -> Graph -> Ranks -> Score
nodeUpdate n graph ranks =
  (1-d) / (fromIntegral dimG) + d * s where
    s = sum [ (nodePass i graph ranks) |
              i <- nodesIn n graph ]
nodePass :: Name -> Graph -> Ranks -> Score
nodePass n graph ranks =
  (nodeRank n ranks) /
  (fromIntegral (outDegree n graph))
```

PageRank algorithm

Example

```
veryClose :: Double -> Ranks -> Ranks -> Bool
veryClose eps cranks pranks =
    foldr (&&) True
    $ zipWith (\ (_,c) (_,p) -> abs (c-p) < eps)
              cranks
              pranks
```

```
stopIt :: Double -> [Ranks] -> Ranks
stopIt eps (pi:ci:nis)
    | veryClose eps pi ci = ci
    | otherwise = stopIt eps (ci:nis)
```

PageRank algorithm

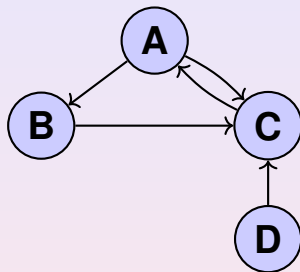
Example

```
ranksUpdate :: Graph -> Ranks -> Ranks
ranksUpdate graph ranks =
    map (\(n,r) -> (n, nodeUpdate n graph ranks))
        ranks
```

```
pageRank :: Double -> Graph -> Ranks -> Ranks
pageRank prec graph ranks =
    stopIt prec
    $ iterate (ranksUpdate graph) ranks
```

PageRank algorithm

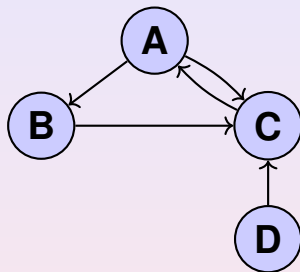
Example



```
g1 = [ ("A", ["C"], 2), ("B", ["A"], 1),  
       ("C", ["A", "B", "D"], 1), ("D", [], 1) ]  
     :: Graph  
r100 = [ ("A", 0), ("B", 1), ("C", 1), ("D", 2) ]  
       :: Ranks
```

PageRank algorithm

Example



```
*Main> pageRank prec g1 r100  
[("A",0.3730180867176842),  
 ("B",0.1960695295812611),  
 ("C",0.3946404722701038),  
 ("D",3.7500000000000006e-2)]
```

Corecursion: dual of recursion

Definition (Corecursion)

Start from a base case and iteratively produce data, based on a recursive relation, in a potentially infinite sequence.

Example (Build a factorials sequence using corecursion)

```
fseq = iterate (\(n,f) -> (n+1, f*(n+1))) (0,1)
```

Trace

```
[(0,1), (0+1, 1*(0+1))] = (1,1), (2,2), (3,6) ...
```

Example (Factorial with corecursion)

```
factorial n = snd (fseq !! n) where  
  fseq = iterate (\(n,f) -> (n+1, f*(n+1))) (0,1)
```

Corecursion: dual of recursion

Example (Fibonacci, once again)

```
fibo n = trd (fiboseq !! n) where
  fiboseq = iterate
            (\(n,f1,f2) -> (n+1, f2, f1+f2))
            (0,1,0)
  trd (_,_,c3) = c3
```

Remark: recursion v corecursion

Recursion progresses towards "smaller" structures; corecursion progresses towards "larger" structures.

Pascal Triangle

Example (Build the triangle via `iterate`)

```
pascalSeq =  
  let  
    infix 6 &:  
    x &: xs = xs ++ [x]  
  in  
    iterate (\l -> zipWith (+) (0:1) (0&:1)) [1]
```

Summary

1. A program can be seen as a network of circular structures which produce and consume data streams
2. `iterate` can be used to implement iterative algorithms. To this end, you have to specify:
 - ▶ a function to "connect" two consecutive iterations
 - ▶ a criterion for stopping the process
3. corecursion: start from a base case and iteratively produce data, according to a recursive relation, in a potentially infinite structure